

memor: architecture and product review

Reviewer framing: product architect, LLM memory and context optimisation. **Date:** 2026-08-22. **Method:** five parallel research agents (architecture, memory quality, competitive landscape, product positioning, context-engineering frontier), plus direct measurement against 2,465 local Claude transcripts (82.5M tokens) and the live memor database (33,970 artifacts, 4,133 recalls, 657 adjudicated recall-artifact pairs).

Every claim cites a `path:line`, a command and its output, or a URL with the date read. Where a number is a lower bound or an estimate, it says so. Two claims in the first draft of this review were wrong; both are corrected in place with the error described, because how they failed is diagnostic.

1. The product being reviewed

memor was conceived as **a common long-term memory layer for every LLM tool on a machine** — one store, so context follows the user between tools, and nothing has to be re-established when they switch.

That is not what the README sells. The README leads with compression percentages, and the first draft of this review consequently graded the product as a compression tool. That was a category error on my part, and it produced a roadmap aimed at the wrong target. This version grades memor against its actual thesis.

The verdict changes completely under the correct framing. As a compression product, memor is capped and near-finished. As a shared memory layer, it is built, working, and structurally unable to prove it.

With one correction to the thesis itself. "One store, many tools" is not a defensible position — Pieces, OpenMemory and Supermemory all occupy it, two of them funded and louder (§6.2). What no competitor has is *how memor writes*: it polls harness transcripts off disk, so a tool contributes memory **without knowing memor exists**. Every rival's write path needs the model to call a save tool, or a hand-built per-harness hook, or settles for screen pixels. That is the line worth defending (§6.3).

1.1 What I had to guess, and what I got wrong

This review was commissioned before the thesis above was stated, and I worked for some time under the wrong one. Since the document's whole claim is that unfalsifiable assertions are worthless, the interpretive debts belong in it.

Guessed and later confirmed wrong: that the README describes the product. It leads with compression percentages, so the first draft graded memor as a compression tool and produced a roadmap headed by constraint pinning — a good feature aimed at the wrong goal. Corrected in full; compression is now Appendix A and constraint pinning is P5.

Guessed and still unverified: that "shared layer" means shared *retrieval*. I have taken the thesis to mean a fact written by one tool should be *retrievable* by another, and measured that. It could also mean shared *budget* (one context allowance across tools) or shared *identity* (one profile of the user's preferences). The measurements here support the retrieval reading only.

Guessed: that the local corpus is representative. All figures come from one machine, one user, mostly one harness — 89% of writes are `claude_code`. The 28.7% cross-tool rate rests on 129 adjudicated pairs in five projects. Directional at best.

Three claims withdrawn during the review, each from reasoning about code rather than running it:

- 1. That recency admits irrelevant-but-fresh memories. The blended threshold cannot do this; `norm_rel` is min-max normalised and the code says so in a comment (`retriever.py:255-260`).
- 2. That the L2/cosine units bug is the main cause of the 40.9% zero-hit rate. Probing the live store showed the gate sits near precision-optimal (§3.2).
- 3. That compaction is the product's central problem. True as a measurement ($\geq 36.9\%$ of typed instructions lost), wrong as a priority for *this* product.

The pattern in all three is the same, and it is the pattern this codebase has been correcting all year: **a number derived from a formula is a hypothesis, not a finding.**

2. Was the shared layer delivered? Yes.

2.1 The store is genuinely tool-agnostic

Scope is by **project**, never by agent (`memor/types.py:34-46`). No agent or source filter exists anywhere in the retrieval path — grepped `memor/retrieve/retriever.py` and `memor/store/sqlite_store.py`; the only `agent ==` comparison is a display-label default at `sqlite_store.py:320`.

Four tools write, four read:

Writers (artifacts)		Readers (recalls)	
<code>claude_code</code>	24,446	<code>claude</code>	3,635
<code>distill</code>	6,930	<code>cursor</code>	427
<code>goose</code>	1,256	<code>codex</code>	38
<code>jcode</code>	1,249	<code>jcode</code>	31
<code>kimi</code>	95		

2.2 Knowledge actually crosses, in both directions

Joining `recall_outcomes` → `recall_log` → `artifacts` gives the ground truth: which tool *received* an artifact that a *different* tool wrote.

<code>claude</code> reads <code>jcode</code> -written artifacts	27
<code>jcode</code> reads <code>claude_code</code> -written	10
<code>jcode</code> reads <code>distill</code> -written	7

Cross-tool share of served artifact-recalls	
Across all projects	5.6% (37 of 657)
In projects where more than one tool is active	28.7% (37 of 129)

The 5.6% is not a design failure, it is under-exercise. Only 3 of the top 14 projects have two tools writing 20+ chunks; 89% of all writes are `claude_code`. Where two tools genuinely share a project, better than one in four served memories came from the other tool.

The mechanism works. The thesis is delivered.

Validated through the public interface, not only from production data. Driving the real `hook_server.handle_request` against an isolated store holding one jcode-written memory, each of `goose`, `jcode` and `claude` received it (§7, P0). Cross-tool retrieval is a working, reproducible property of the system, not an artifact of how the production rows happen to be joined.

2.3 The differentiator is passive capture, not the shared store

The shared store on its own is **not** a defensible niche. Mem0's OpenMemory MCP Server (mem0.ai/blog/introducing-openmemory-mcp, read 2026-08-22) is explicitly "a private, local-first memory server that creates a shared, persistent memory layer for your MCP-compatible tools", markets exactly this scenario — *"Define technical requirements in Claude Desktop. Build in Cursor. Debug in Windsurf"* — and supports Cursor, Claude Desktop, Windsurf and Cline. Local, free, and the same headline claim.

The difference is in **how memory gets written**, and it is architectural.

OpenMemory exposes `add_memories` as an MCP tool. Something must *decide to call it*: the agent, prompted to save, or the user. That is opt-in capture, and it inherits the failure mode of every manual knowledge base — the discipline lapses and the store goes stale.

memor never asks. The daemon polls agent transcript stores directly on disk every 30 seconds (`memor/daemon.py:23`, `memor/ingest/sources.py:180–188`): Claude's `projects` dir, jcode sessions, Goose's SQLite database, Kimi's session files. Its MCP surface is **read-only** — `memor_recall` and `memor_retrieve`, with no write tool (`memor/proxy/mcp_retrieve.py:3–6`).

The consequence is the interesting one: **a tool contributes memory without knowing memor exists.** Goose wrote 1,256 artifacts and has never called memor. No plugin, no MCP registration, no prompt asking the model to save anything. That is a materially different bet from OpenMemory's, and it is the part a competitor cannot copy without also writing a parser per harness.

It also explains the asymmetry in §2.1: four tools write, but writing requires only a transcript on disk, whereas reading requires an integration. That is why `goose` and `kimi` appear as writers and never as readers.

The honest risk. Passive capture is why 46.3% of stored tokens are ephemeral scaffolding (§3.3) — nothing decided what was worth keeping. OpenMemory's opt-in write is a curation filter that memor pays for in noise. P2 exists to close that gap without giving up the property that makes the approach distinctive.

2.4 What that is worth, and the ledger nobody drew

The product's own numbers, on one line:

```
compression saved : 2,453,627 tokens
recall injected   : 1,146,846 tokens   (cost)
-----
net               : 1,306,781 tokens
```

Recall spends 47% of what compression saves. That is a coherent architecture — compression funds the memory layer — and it is a far better description of the product than "11.7% saved on tool output". But it only nets out if a recalled memory is worth more than the ~469 tokens it costs to inject.

The mechanism is plausible on arithmetic: mean recall injection is 469 tokens, while a single `Read` of a 500-line file is roughly 6,000. A recall that averts one file re-read is ~13× cheaper than the derivation it replaces.

Whether recalls actually replace derivations is unmeasured, and that is the central open question of the product, not a detail.

3. Why it looks like a compression product

Because **compression is the only half that can report**.

3.1 The quality loop is hardcoded to one tool

`memor/daemon.py:367–372` :

```
# Feedback + turn metrics: Claude transcripts only
for unit in pending:
    if unit.agent != "claude" or unit.path is None:
        continue
```

That single guard clause gates **both** of `memor`'s measurement systems.

`analyze_session_feedback` (`daemon.py:385`) is the only writer of `use_count`, and `correlate_with_recalls` (`daemon.py:396`) is the only writer of `had_recall`. Neither runs for `jcode`, `cursor`, `goose` or `kimi`. The consequences are exact and visible:

	used	rejected	unused	pending
same-tool recalls	170	0	354	96
cross-tool recalls	0	0	0	37

Every one of the 37 cross-tool recalls — the product's headline capability — sits `pending` forever. Same-tool recalls are adjudicated and score **100% of judged as used**.

The second system is degraded further: only **48 of 3,620 `session_stats` rows (1.3%)** carry a non-zero `recall_turn_count`, against 4,133 logged recalls. Two causes compound. The Claude-only guard is one. The other is that `correlate_with_recalls` matches on `session_id` (`memor/turn_metrics.py:95–99`), and **100% of `jcode` recalls record no `session_id` at all** (31 of 31), so even lifting the guard would not correlate them without also fixing the identifier.

The one feature memor exists for is the one feature it cannot measure. That is why a rigorous, measurement-driven project drifted into describing itself by its compression percentage: that number is the one the instrumentation produces.

This is the same failure class the project has corrected seven times in compression — a claim that cannot be falsified — except here it is load-bearing for the entire product thesis.

3.2 The relevance gate is calibrated in the wrong metric

This is the clearest code defect found. It is a units error rather than a mis-tuning, and its practical cost turned out to be smaller than the conversion table alone suggests — the measurement that qualifies it is below, and it reversed my initial reading.

`vec_artifacts` is declared with no distance metric (`memor/store/sqlite_store.py:135–136`), so `sqlite-vec` uses its default, **L2**. Verified from first principles: two orthogonal unit vectors (cosine 0.0) return distance $1.4142 = \sqrt{2}$. The store then computes `sim = 1.0 - float(r["distance"])` (`sqlite_store.py:507`).

1 – L2 is not a cosine. On unit vectors $L2 = \sqrt{(2-2\cos)}$:

true cosine	stored sim	vs the –0.05 gate
+0.90	+0.553	pass
+0.70	+0.225	pass
+0.50	+0.000	pass
+0.40	–0.095	blocked
+0.30	–0.183	blocked
+0.00	–0.414	blocked

The gate at `min_similarity = –0.05` (`memor/recall.py:116`, applied `retriever.py:263`) carries a comment stating the intent: *"Static embeddings put relevant content at >0 and noise at <0, so the default floor is 0.0."* Correct **for cosine**. Applied to **1 – L2**, the effective cut is at **true cosine $\approx$ +0.449** — so everything between 0.0 and 0.45 is discarded before fusion, before the memory lane, and before any blended score exists.

That band is exactly where cross-tool matches live. A memory written by jcode about the same subsystem, in different words, is related-but-not-duplicate.

But the gate is better calibrated than the unit error suggests, and this qualifies the finding substantially. Converting the store's `sim` to true cosine and probing the live store with five on-topic queries (150 candidates) against four nonsense queries (112 candidates):

cosine cut	on-topic admitted	nonsense admitted	precision
0.50	11%	0	100%
0.449 (current gate)	~41%	2	~97%
0.40	67%	18	85%
0.35	81%	22	85%
0.30	91%	28	83%
0.25	100%	53	74%

So `-0.05` in L2 units lands at cosine 0.449, which is close to the precision-optimal cut for this embedder. **Whoever tuned it tuned it on behaviour, and arrived somewhere defensible by feel.** The bug is real — the constant means something different from what its comment claims, and cannot be reasoned about or ported to another embedder — but it is not costing the recall disaster the mapping table alone implies.

What it *is* costing is the 0.40–0.449 slice: roughly **26 percentage points of recall for 16 additional false admits**. On this sample the blocked-but-on-topic items are visibly relevant:

```
cos +0.444  BLOCKED  'if the competitors are claiming 20% savings...'
cos +0.429  BLOCKED  'Why are we not able to improve our compression...'
cos +0.421  BLOCKED  'wait so you did identify the reason compression...'
```

The apparent signal-to-noise separation on raw `sim` is only ~ 0.16 , which looks like a weak embedder. It is not: under true cosine, `potion-base-8M` separates matched from nonsense pairs by **0.573**. The model is fine; the unit conversion compresses the usable range into a band too narrow to reason about.

Correction to this review's first draft. I initially reported that the 0.3 blended threshold admits irrelevant-but-fresh memories, since a zero-relevance item scores $0.25 \times 1.0 + 0.10 \times 0.5 = 0.300$. Arithmetically true, practically irrelevant: `norm_rel` is **min-max normalised over the candidate set** (`retriever.py:330`), so the top candidate always scores 1.0. The code says so in a comment I should have read first (`retriever.py:255–260`): *"Min-max normalization forces the top hit to a normalized score of 1.0 ... Gating on raw cosine here is the real relevance filter."* The design anticipated the objection and answered it. The remaining defect is that the filter it points to is in the wrong units.

Predicted and confirmed. If the raw gate is what rejects, then `no_hits` rows should show an empty candidate set rather than a filtered one. All 1,543 `no_hits` rows have `top_score = 0.000` exactly, with no intermediate values.

What this does not explain. Since the gate sits near the precision-optimal cut, it cannot be the whole cause of 40.9% zero-hit. The residual is most likely §3.3: 88% of artifacts have never been recalled, and a store full of scaffolding has little to return however the gate is set.

3.3 What is stored is mostly not knowledge

Of 6,948 memory artifacts (5.67M tokens), **46.3% of tokens are provably ephemeral scaffolding**: 2,316 subagent task prompts ("You are implementing Task 11..."), 293 status reports, plus caveat banners. Sampling the remainder shows more work-log.

The signal that matters: **52.4% of memories ever recalled carry rationale language** ("because", "instead of", "turned out", "root cause") against 44.8% of the corpus. Retrieval is already selecting for rationale — the class that cannot be recovered by reading the repo, and the only class worth carrying between tools.

Anthropic reached the same conclusion independently. Claude Code's auto-memory "**skips anything it can derive from the codebase, such as architecture, file paths, or debugging fixes**", storing `project` memories only for decisions "*that Claude can't derive from the code or git history*". They encoded the thesis as a write-filter. `memor` has no such filter, and **88.0% of its artifacts (29,909) have never been recalled once**.

4. What is genuinely excellent

4.1 The measurement discipline

Seven estimates collapsed under measurement in a single session: diff compressibility 39%→0.1% realised, session-start 30%→4.5%, fetched pages 40%→0.10%, classifier release 5.39%→0.10%, duplicate elision 1.78%→0.53%, superseded reads 15.77%→0.79%, and a "code-safe" crusher that reported **zero** loss until independent predicates showed it still dropped 8,371 code lines (`docs/receiving-compressor-bottleneck.md`).

Four documents in `docs/` are retractions of the author's own claims. No competitor does this. `Mem0` publishes 26% over OpenAI on LOCOMO; Zep publishes 75.14% and argues the benchmark is flawed; Honcho claims 60–90%. Zep's sharpest observation is that **Mem0's own paper shows a plain full-context baseline beating Mem0** (~73% vs ~68%) — the standing warning for every memory product, `memor` included.

4.2 The safety architecture

The source guard is correct, and four attempts to weaken it this session failed on evidence. Releasing guarded payloads destroys 5,230 code lines across 273 of 443 payloads, because `sed -n '1,200p' file.tsx` is a file read that merely lacks line numbers.

The system fails open almost everywhere, right for a sidecar: socket down falls back inline (`bin/memor-hook.py:80–85`), missing embedder returns the body unchanged (`memor/proxy/memory.py:115–120`).

4.3 Concurrency

WAL plus a 30-second busy timeout (`memor/store/sqlite_store.py:106`), with the motivating incident in the comment: 356 "database is locked" errors in one session under Python's 5-second default. This is what makes one store shared by several concurrently-running tools viable at all — a precondition for the thesis.

5. Other defects

5.1 Finished work stranded on unreachable tags

Four shelved tags, **1,759 insertions**, contained in no branch:

Tag	Commits	Diff
shelved/temporal-validity	11	19 files, +699
shelved/reaffirmation-recency	4	9 files, +532
shelved/dashboard-service	1	4 files, +352
shelved/distillation-quality	2	3 files, +176

temporal-validity matters most: the disputes table and validity column exist in the live database — disputes holds 0 rows, validity is 1.0 across all 4,061 rows, and **neither string appears anywhere in the Python source**. A fossil of a completed feature. Staleness is the known failure mode of code memory, and it compounds across tools: a memory written by one tool and read by another months later has no freshness signal at all.

5.2 Dead code and a silent kill switch

memor/proxy/output_shaper.py is **271 lines referenced from nowhere**.

memor/proxy/shim.py:35–46 sets compressor_ready = False on any exception, and prepare_request_body checks that latch on entry. **One transient DB lock permanently disables compression for the life of the process**, no reset, no alert.

5.3 The benchmark measures the wrong construct

eval-longmemeval reports 95.0% any-hit, but scores **set membership of session IDs** (memor/eval/longmemeval.py:111), and the file's own docstring admits "any-hit overstates how many questions the agent could actually answer". LongMemEval is conversational personal-assistant memory ("I have a dog named Max"), not coding memory, and certainly not *cross-tool* coding memory. The one end-to-end number in the repo collapsed from **63.8% to 8.6%** when the harness was corrected to call production recall() (CHANGELOG.md:39).

6. Strategic position

The platform is not the threat. The other startups are, and the position is **more crowded than the thesis assumes**.

6.1 The platform leaves this gap open

Anthropic has shipped context editing (2025-09-29), the memory tool (memory_20250818), auto-compact, and auto-memory. Against a *compression* product that is close to fatal: they report 84% token reduction on a 100-turn eval, and a user will compare that to 11.7% and stop reading.

Against a *cross-tool memory layer* it is not competitive, because **no model provider will make its memory work well inside a rival's harness**. Anthropic's memory is Claude's. Cursor's Memories are Cursor's. Anthropic ships the memory *protocol* and states plainly that "memory lives entirely in your application" — the socket, not the appliance.

The economics also stop being embarrassing. On a Max subscription compression saves **\$0.00**; cross-tool memory claims to save *re-establishment*, which is felt on every tool switch regardless of billing model.

6.2 But three products already occupy this exact position

Product	Shared store?	Local-first?	Markets cross-tool?
Pieces (PiecesOS)	Yes, one LTM store over MCP to ~20 harnesses	Yes, on-device	Yes, as the architecture
OpenMemory (Mem0)	Yes, explicitly	Yes, but needs Docker + an OpenAI key	Yes, headline
Supermemory	Yes	Cloud-first; self-host on paid tiers	Yes, headline
claude-mem	Yes by construction (one SQLite, project column, no agent column)	Yes	No — never claims it

(All read 2026-08-22: mem0.ai/blog/introducing-openmemory-mcp, supermemory.ai/personal, docs.pieces.app/products/mcp/get-started, docs.claude-mem.ai/architecture/database.md.)

Zep, Cognee, Letta and Honcho are **not** competitors here: they are memory backends for developers building their own agents, scoped per user or session inside one application, not across a developer's tools.

"One store, many tools" is not a pitch. Two funded companies already say it louder. The uncontested intersection is narrower — local **and** project-scoped **and** no cloud **and** no Docker or heavyweight daemon — and those are implementation constraints, which are easier to copy than to defend.

6.3 What is actually hard to copy

Two things, and neither is the store.

Passive artifact capture. Every competitor's write path needs cooperation from something. Researched across the four closest products, the taxonomy has four cells and memor is alone in its own:

Approach	Products	Needs the model to comply?	Needs a per-harness adapter?	Gets structured data?
Model-invoked write tool	OpenMemory (<code>add_memories</code>), Supermemory one-click save	Yes	No	Yes
Harness-cooperative hooks	claude-mem, Supermemory plugins	No	Yes	Yes
Ambient screen capture	Pieces (Vision/Clipboard/Audio)	No	No	No — pixels
Artifact polling	memor	No	No ¹	Yes

¹ It needs a *parser* per harness, but not the harness's cooperation.

The distinctions are load-bearing. OpenMemory's docs recommend installing skills and pasting a starter prompt "so the assistant knows to use it" (docs.mem0.ai/vibecoding) — memory contingent on prompt engineering, and if the model does not think to save, nothing is saved. claude-mem is "fundamentally a hook-driven system" whose installer asks which IDEs to wire up (docs.claude-mem.ai/hooks-architecture.md, /installation): automatic at runtime, but only inside a harness someone has built an adapter for, and only if that harness exposes hooks at all. Pieces captures Vision, Clipboard and Audio — a closed list (docs.pieces.app/products/core-dependencies/pieces-os/long-term-memory) — so it sees the *pixels* of a coding agent, misses anything headless, SSH'd or scrolled off screen, and gets OCR-grade text with no session ID or tool-call boundary.

memor reads the transcripts harnesses already write to disk for their own reasons. **A tool contributes memory without knowing memor exists.** Goose wrote 1,256 artifacts having never called it, and none of the four competitors can produce that demo.

Two honest caveats. This is a moat of *engineering*, not architecture: per-harness format reverse-engineering that breaks silently when Claude Code changes its transcript layout, defensible only while the maintenance tax is paid, and copyable by a funded competitor in a quarter. And Pieces' capture list is documented-absence, not confirmed-negative — its source list beyond the three modalities could not be verified.

Falsifiable measurement. No competitor ships tools letting a user disprove its claims on their own traffic. Narrow appeal, invisible to most buyers, genuinely unmatched.

If the product repositions, this is the line — not the shared store: *contribute memory without knowing memor exists.*

6.4 Demand is validated by supply, not by users

At least eight entrants in twelve months (Bindly, RedPlanetHQ CORE, Drift, Jumbo, and others), **every one scoring 1–4 points on Hacker News**. Many people are building this fix; few are organically complaining about the problem. That asymmetry should temper any assumption that cross-tool memory is a felt pain with buyers waiting.

Standardisation is also moving: every serious player ships an MCP memory server, so the plumbing is commodity. AGENTS.md is the precedent — 60k repos, 25 agents, now under the Linux Foundation's Agentic AI Foundation. Cross-harness context standardised quickly once a neutral body took it. A shared-memory *protocol* could commoditise the same way, which would leave capture quality and curation as the only durable ground.

(HN sampling via hn.algolia.com, 2026-08-22. Reddit returned 403 and was not sampled, so this demand picture is HN-only.)

7. Ranked backlog

Ranked by (impact on the shared-layer thesis × confidence).

P0 — Make cross-tool recall measurable

What. Extend the feedback loop past Claude. `memor/daemon.py:372` skips every non-Claude session, so neither `analyze_session_feedback` nor `correlate_with_recalls` ever runs for jcode, cursor, goose or kimi.

Scoped by an end-to-end test rather than by reading the code, which changed the answer. Driving the real `hook_server.handle_request` against an isolated store seeded with a jcode-written memory:

```
goose    recall -> memory delivered: True
jcode    recall -> memory delivered: True
claude   recall -> memory delivered: True
all three verdicts: pending
```

Cross-tool *retrieval* is confirmed working through the public interface. Then, feeding the analyzer a goose recall plus a transcript in which the assistant reuses the memory:

```
before: [('jc-1', 'pending')]
analyze_session_feedback -> 1
after : [('jc-1', 'used')]
```

The analyzer is already agent-agnostic. It resolved a goose recall to `used` and wrote `use_count`. Nothing in the scoring logic is Claude-specific.

But the reader that feeds it, `_extract_stamped_texts` (`feedback.py:245`), returns **0 assistant and 0 user texts** on a real jcode journal — jcode writes `{append_messages, meta}` records, and the reader expects Claude's `{type, message}` shape with ISO timestamps. On a real Claude transcript the same function returns 4,584 and 1,230. `memor/ingest/jcode.py:94` parses the jcode file into 28 messages.

A second obstacle sits behind that one: `analyze_session_feedback` takes a `transcript_path: Path`, and **Goose has no per-session transcript file at all** — its sessions live in SQLite. So the signature assumes a file-per-session model that one supported agent does not use.

So P0 is three things, not one:

1. Remove the `agent != "claude"` guard (`daemon.py:372`).
2. Give the feedback path a per-agent reader, and an interface that does not assume a file per session.
The reading logic already exists in `memor/ingest/jcode.py:94`, `goose.py` and `kimi.py`; the work is reuse plus a source abstraction, not new parsing.
3. Populate `session_id` on the jcode recall path — 100% of jcode recalls record none (31 of 31), and `correlate_with_recalls` joins on it (`memor/turn_metrics.py:95–99`).

Had this been scoped from the code alone it would have been filed as "delete a guard clause" and failed silently on step 2.

Noticed while testing, and worth fixing separately. `feedback._extract_texts` and `_extract_assistant_texts` (`feedback.py:41,69`) are dead — nothing on the live path calls them — and `_extract_texts` still matches `type == "human"`, which never appears in a transcript. That is the exact bug

the module's own docstring records as fixed (`feedback.py:228–230`). The fix landed in the replacement; the original was left behind.

Why first. All 37 cross-tool recalls are `pending` , and only 1.3% of `session_stats` rows carry a recall correlation. Until this lands, every statement about the product's core capability is unfalsifiable — including the favourable ones in this document. The project's own history says an unfalsifiable claim is worth nothing: seven collapsed this session, and three claims in this review had to be withdrawn for the same reason.

Success metric. Cross-tool `used / rejected` verdicts become non-zero, and the cross-tool use rate is comparable against the same-tool 100%-of-judged baseline.

Effort. Small-to-moderate, and better understood now: one guard clause, a transcript-reader indirection reusing existing parsers, and a session-id fix.

Risk. Low. It adds measurement, changes no retrieval behaviour.

P1 — Express the relevance gate in cosine, then tune it on evidence

What. Declare `distance_metric=cosine` on `vec_artifacts` , or convert with `cos = 1 - L22/2` , so the constant means what its comment says. Then decide the floor from the precision/recall table in §3.2 rather than by feel.

Why. Two reasons, and the second is the smaller one.

First, correctness of reasoning: `min_similarity = -0.05` currently denotes cosine 0.449, so nobody can reason about it, port it to another embedder, or change it safely. That is a latent hazard every time the embedding model moves.

Second, recall: moving the cut from 0.449 to ~0.40 admits **67% of on-topic candidates instead of 41%**, costing 16 additional false admits on the probe set (85% precision, down from 97%). That is the specific trade available.

Important qualification. The measured table shows the existing gate is close to precision-optimal for `potion-base-8M` . This is a units bug and a modest recall opportunity, **not** the cause of the 40.9% zero-hit rate — see §3.3 for the more likely cause. I over-claimed this in an earlier draft on the strength of the conversion table alone, before probing the live store.

Success metric. Zero-hit rate on stores with >2,000 artifacts falls from 32.1%, **and** the P0 cross-tool use rate does not fall. Both must hold.

Effort. Small in code; needs a `vec_artifacts` re-index migration.

Risk. Real. Chroma's context-rot work (2025-07-14, 18 models, 194,480 calls) finds distractor harm is the dominant degradation lever and grows with context length, so the 16 extra false admits are not free. **Do not ship before P0 can measure it.**

P2 — A write-side curation filter

What. Refuse to store what the repo already answers. Prefer rationale, failed attempts, corrections, preferences.

Why. 46.3% of memory tokens are ephemeral scaffolding; 88% of artifacts are never recalled; recall already selects for rationale at 52.4%. Anthropic shipped exactly this filter. It also improves the P1 risk profile: a cleaner store means a wider gate admits knowledge rather than noise.

Success metric. Never-recalled share falls from 88%; rationale share of stored tokens rises from 44.8%.

Effort. Small-to-moderate; `_signal_score()` (`memor/ingest/claude_code.py:82`) is the hook point.

P3 — Cross-tool acceptance test

What. An end-to-end test: write a decision in tool A, recall it in tool B, assert it is served and used. Run it for each supported pair.

Why. The product's headline capability has no acceptance test. 28.7% is measured from incidental production data, not from anything guaranteeing the path keeps working. Nine integration surfaces across three mechanisms means silent breakage is the default failure mode.

Effort. Moderate.

P4 — Detect ingest drift

What. Alert when a configured source stops producing artifacts. Per-source "last successful ingest" plus a staleness threshold, surfaced in `memor service status` and the dashboard.

Why. §6.3 identifies passive artifact capture as the one genuinely hard-to-copy property, and its stated weakness is that it **breaks silently**. Confirmed by experiment rather than assumed: feeding `parse_session` a transcript whose top-level key was renamed from `messages` to `turns` yields **0 artifacts, no exception and no warning**, while the current format yields 1. Searched `memor/ingest/` and `memor/daemon.py` for staleness or drift detection; **none found**. Today the failure presents as "memory got worse over the last month" with no diagnosis.

This is cheap insurance on the moat. A moat maintained by reverse-engineering needs a tripwire when the terrain moves.

Success metric. A deliberately corrupted source directory produces a visible warning within one poll cycle.

Effort. Small. The daemon already tracks per-source state in `~/.memor/ingested.json`.

P5 — Unshelve temporal validity

What. Restore `shelved/temporal-validity` (+699) or delete the orphaned schema.

Why. Staleness compounds in a shared layer: a memory written by one tool and read by another months later has no freshness signal. The work is done.

P6 — Constraint pinning (was P0 in the first draft)

What. Detect user-issued constraints, persist them, re-inject after compaction.

Why it moved. Measured on this machine, **≥36.9% of the user's own typed instructions are lost to compaction** (433 of 1,172 across 63 events), and 81.5% of sessions over 2MB compact. The literature is stronger still: COMPINT (arXiv:2608.11242) measures 17% constraint retention and closes it to >90% with a

sidecar; ConstraintRot (arXiv:2606.22528) shows violations going 0%→30% purely from compaction, restored to 0% by pinning.

This is a real and valuable feature, and under the compression framing it was correctly P0. Under the shared-memory framing it is adjacent: it improves memory *within* a session rather than *between* tools. Build it after the core thesis is measurable.

Caveat on the 36.9%. It is a lower bound. The retention test counts a constraint as surviving when 60% of its content words appear in the summary, and inspection shows that fires on stopwords where the summary merely discusses the topic. A first pass also counted 21,421 "constraints" that were skill-file boilerplate; restricting to human-typed turns cut it to 1,172.

P7 — Repair operational faults

Remove or reset the `compressor_ready` latch (`memor/proxy/shim.py:35–46`); delete `memor/proxy/output_shaper.py` (271 dead lines); delete `feedback._extract_texts` and `_extract_assistant_texts` (`feedback.py:41,69`, dead and carrying a known-fixed bug); extract `memor/install/` from `memor/cli.py` (1,871 lines, install logic for seven agents at 701–1102).

8. What not to build

- **More compression.** Four independent attacks measured negative in one day; the remaining 4.04% is source code. `79702be` records this.
 - **A better content classifier.** Worth 0.03%–4.04% depending only on which compressor receives the release; never the bottleneck.
 - **More integration surfaces.** Nine agents, three mechanisms, two non-identical support matrices. Depth on the pairs that are actually used beats breadth — and P3 says the existing pairs are not even tested.
-

9. Open questions

1. **Do recalls replace derivations?** The 13× arithmetic (469 tokens injected vs ~6,000 to re-read a file) assumes the recall averts the read. Unmeasured, and it is the product's core economic claim.
 2. **Does cross-tool recall help as much as same-tool?** P0 exists to answer this. It could plausibly be *better* (the other tool's knowledge is genuinely new) or *worse* (different conventions, stale context).
 3. **Is 28.7% good?** No baseline exists. It could be near a ceiling set by how often two tools genuinely need the same knowledge.
 4. **Would fixing the gate improve answers or just add distractors?** The measured trade is 41%→67% recall for 97%→85% precision. Which side wins is unknown without P0.
 5. **Is the local ≥36.9% constraint loss representative?** One user, one harness, 63 events. COMPINT reports far worse across compactors.
-

Appendix A: the compression finding, for completeness

Compression is honest, finished, and capped. On the local corpus the log crusher earns **88% of all realised savings** at a 70.2% rate; everything else combined is 1.2%. Four attacks on the remaining 4.04% all measured negative (docs/receiving-compressor-bottleneck.md): a middle-ground compressor is worth 0.52% against an existing 0.21%; releasing guarded payloads destroys 5,230 code lines; a command allowlist cannot see `find -exec cat` or heredocs; and pinning code lines reported zero loss only because it was measured with the pinning regex.

The one change that survived was lowering the floor from 2,000 to 1,000 characters (+0.57% of tool-result tokens, no answer-critical line lost).

Under the shared-memory thesis, compression's role is clear and worth keeping: **it funds the memory layer**. It saves 2.45M tokens against recall's 1.15M cost. That framing is defensible in a way that "11.7% saved" is not.

Appendix B: what was measured, and how

Claim	Method
Store is tool-agnostic	<code>Scope</code> has no agent field (<code>types.py:34–46</code>); grepped retrieval path for agent filters
Cross-tool retrieval works	End-to-end through <code>hook_server.handle_request</code> on an isolated store: a jcode-written memory delivered to goose, jcode and claude
Feedback analyzer is agent-agnostic	Fed a goose recall plus a reusing transcript; resolved <code>pending</code> → <code>used</code> and wrote <code>use_count</code>
jcode transcripts unreadable by feedback	<code>_extract_stamped_texts</code> (the live reader) returns 0/0 on a real journal, against 4,584/1,230 on a real Claude transcript; <code>ingest/jcode.py:94</code> reads the jcode file as 28 messages
Ingest drift is silent	Five drift modes — renamed top-level key, renamed roles, renamed content key, restructured blocks, dict instead of list — each yield 0 artifacts, no exception, no warning
5.6% / 28.7% cross-tool	Joined <code>recall_outcomes</code> → <code>recall_log</code> → <code>artifacts</code> , comparing reader agent to writer source
All cross-tool recalls pending	Same join, grouped by <code>outcome</code>
Feedback is Claude-only	<code>memor/daemon.py:367–372</code>
L2-vs-cosine gate	Orthogonal unit vectors through sqlite-vec return $\sqrt{2}$, confirming L2; table from $L2 = \sqrt{(2-2\cos)}$
Gate precision/recall	5 on-topic queries (150 candidates) vs 4 nonsense (112) against the live store, converted to true cosine
Embedder is not the problem	Matched vs nonsense pairs under true cosine: 0.573 separation on <code>potion-base-8M</code>
Token ledger	<code>sum(tokens_injected)</code> from <code>recall_log</code> vs <code>proxy_savings</code> before/after
46.3% ephemeral / 52.4% rationale	Regex classification over all 6,948 memory artifacts
≥36.9% constraint loss	Human-typed constraints from turns preceding 63 compactions, tested for 60% content-word survival; boilerplate excluded
81.5% of >2MB sessions compact	2,465 transcripts scanned for compaction markers, bucketed by size
Shelved work	<code>git tag</code> , <code>git branch --contains</code> , <code>git diff --shortstat</code>

Research agents: `arch-map2` , `memory-qual2` , `landscape` , `product-pos` , `ctx-frontier` . Full reports in `docs/competitive-landscape-2026-08.md` and `docs/research/context-quality-2026-08.md` .

Appendix C: research provenance

Six research agents were run. The first five were commissioned under the compression framing and are reported here only where their findings survived re-framing:

Agent	Question	Fate of its conclusions
arch-map2	Structural audit, entry points, failure modes	Survives; framing-independent
memory-qual2	Write path, read path, feedback loop, benchmark validity	Survives; the shelved-tag and threshold findings are load-bearing
landscape	Competitors and platform absorption	Partly survives. Independently reached "lead with cross-harness memory", but its competitor table did not distinguish shared stores from per-tool silos
product-pos	Value proposition and effort allocation	Survives. Reached "compression should not be the headline" from the effort histogram alone
ctx-frontier	Compaction integrity, context rot	Survives as evidence; its recommendation (compaction is the top priority) was correct under the wrong thesis and is now P6
crosstool	Does anyone else genuinely share one store across tools, and how does each capture?	Commissioned after the thesis was corrected. Produced §6.2 and §6.3, the two findings that changed the strategic conclusion

The sixth agent existed only because the thesis was corrected. Its finding — that the shared store is crowded but the capture mechanism is not — reversed the strategic section. Had the review shipped at its first draft, it would have recommended a compaction feature for a cross-tool memory product, and called a crowded position defensible.